

Algoritmos de Ordenamiento

Alexa Alpizar Mora, C20281, alexa.alpizar@ucr.ac.cr

Resumen—En este trabajo se recolectó y evaluó la duración de ejecución de los algoritmos de ordenamiento, en este caso; por selección, inserción, mezcla, rápido, por montículos y residuos. Para esto, mediante experimentos con arreglos de diferentes tamaños y de números aleatorios se tomaron los resultados para representarlos gráficamente. El resultado fue el esperado, se mostraron en los gráficos diferencias en la complejidad temporal de los algoritmos y el comportamiento de los algoritmos cuando se utilizan conjuntos de datos de diversos tamaños. Se concluyen las diferencias y eficiencia según el objetivo previsto.

Palabras clave—ordenamiento, selección, inserción, mezcla, rápido, montículos, residuos, arreglo.

I. INTRODUCCIÓN

Los algoritmos son un conjunto de instrucciones definidas, ordenadas y acotadas para resolver un problema, realizar un cálculo o desarrollar una tarea, es decir, un procedimiento por medio de una serie de pasos ordenados para la solución de una situación y así conseguir un fin (Cortez Galván, 2024) [1]. Existen diferentes algoritmos de ordenamiento, los cuales se clasifican en dos grupos, por comparaciones y en tiempo lineal (Peña et al., 2010) [2]. En el caso de este proyecto específicamente, del primer grupo se experimentará con: *Insertion Sort* que toma uno de los elementos de la lista y lo inserta en la posición correcta y repite el proceso hasta que todos los elementos están ordenados, *Merge Sort* que sigue el principio de “divide y conquista”, dividiendo repetidamente la lista en mitades hasta que son lo suficientemente pequeñas para ordenarlas y luego las une ordenadamente, *Heap Sort* que utiliza una estructura de datos de montículo para organizar los elementos de la lista y colocarlos en un montículo para luego extraer uno por uno y así obtener la lista ordenada, por último *Quick Sort* que elige un elemento como pivote y divide la lista en dos sublistas alrededor del pivote para luego ordenarlas recursivamente. Por otra parte, el segundo grupo con Radix Sort; adecuado para clasificar números enteros dentro de un rango específico que cuenta el número de ocurrencias de cada valor en la lista y luego reconstruye la lista ordenada. Finalmente, el objetivo principal es implementar los algoritmos de ordenamiento vistos en el curso y recolectar información sobre el desempeño real de los mismos. Luego, analizar resultados y reflexionar sobre su eficiencia teórica, así como de la observada.

II. METODOLOGÍA

Para lograr lo propuesto, se implementaron los algoritmos por Selección (Selection Sort), Inserción (Insertion Sort), Mezcla (Merge Sort), Montículos (Heapsort), Rápido (Quicksort), y Residuos (Radix Sort) por medio del lenguaje de programación C++. Se programaron en funciones de la clase Ordenador. Para

«Ordenador.hpp», se tomó como base la plantilla proporcionada por el profesor, en la cual se mantuvieron los encabezados de funciones establecidas y se agregaron los métodos privados necesarios para que cada algoritmo ordenara arreglos en orden ascendente. El código se desarrolló basado en los pseudocódigos que se exponen en los libros “Data Structures and Algorithms” de Aho, Hopcroft y Ullman [3], así como en “Introduction to Algorithms” de Cormen, Leiserson, Rivest y Stein [4]. Seguidamente, se crearon los arreglos de tamaño 1000, 10000, 100000, 1000000 de enteros positivos seleccionados al azar y se ejecutó cada uno de los algoritmos al menos tres veces con cada uno de los arreglos. El tiempo de duración en segundos de cada ejecución se tomó y registró junto con sus respectivos promedios (se imprimieron en la consola). Para cada uno de los algoritmos, se creó un gráfico con los tiempos promedio en cada eje Y, además del tamaño del arreglo en los ejes X, luego de haber organizado los datos en una tabla para realizar su análisis posteriormente. Por otra parte, se llevó a cabo otro gráfico, esta vez con todos los tiempos promedio de ejecución juntos. Finalmente, el experimento se documentó en formato de reporte de investigación para exponer el planteamiento del problema, diseño experimental, análisis de resultados y conclusiones.

III. RESULTADOS

Los tiempos de ejecución de las tres corridas de los algoritmos según el tamaño de cada arreglo y su respectivo promedio se muestran en el cuadro I.

Cuadro I Tiempo de ejecución de los algoritmos

Tiempo (s)					
Algoritmo	Tam. (k)	Corrida			Prom.
		1	2	3	
Selección	1000	0.0018292	0.0019003	0.0018332	0.0018543
	10000	0.124513	0.108019	0.11662	0.116384
	100000	10.2487	11.6522	12.9707	11.6239
	1000000	2570.83	1284.8	1088.82	1648.15
Inserción	1000	0.0011974	0.0012377	0.0013154	0.0012502
	10000	0.177675	0.129499	0.0940655	0.133747
	100000	11.5751	14.4776	14.3647	13.4725
	1000000	1053.6	1276.16	1296.65	1208.8
Mezcla	1000	0.0001856	0.0001669	0.0001676	0.0001734
	10000	0.0020293	0.0020349	0.0026146	0.0022263
	100000	0.0339975	0.0290231	0.0252923	0.0294376
	1000000	0.29726	0.384579	0.383467	0.355102
Montículos	1000	0.0002537	0.0002672	0.0002607	0.0002607
	10000	0.0037114	0.0040288	0.0041972	0.003979
	100000	0.0514923	0.045323	0.0438518	0.0468897
	1000000	0.589567	0.570954	0.579311	0.579944
Rápido	1000	0.0001195	0.0001161	0.0001122	0.0001193
	10000	0.0024524	0.0024842	0.0025155	0.0024840
	100000	0.144332	0.159071	0.145594	0.149666
	1000000	14.2511	18.3022	20.5532	17.7022
Residuos	1000	0.0002301	0.0001055	0.0000875	0.0001411
	10000	0.0007398	0.0006696	0.0006097	0.0006731
	100000	0.0059406	0.0047528	0.0046147	0.0051027
	1000000	0.0490721	0.0390723	0.0636916	0.050612

Para el algoritmo de ordenamiento por Selección, el resultado fue una variabilidad significativa en los promedios de tiempo de ejecución entre distintos tamaños de arreglo. Por ejemplo, para un tamaño de 1,000,000 elementos, el promedio fue de 1648.15s, mientras que para otro de 1000 elementos fue de 0.0018543s. Esta tendencia de variabilidad también se mantuvo en las corridas. A pesar de ello, cada corrida fue relativamente consistente en comparación de las demás, con excepción de la ejecución 1 con un millón de elementos. Para el algoritmo de ordenamiento por Inserción, también se observa variabilidad en los promedios de tiempo de ejecución entre distintos tamaños de arreglo. Los rangos de tiempo varían significativamente entre más elementos. Los tiempos entre corridas de un arreglo del mismo tamaño son más consistentes entre ellos, que en el caso anterior por selección. Por otro lado, para el algoritmo de ordenamiento por Mezcla, los tiempos promedio de ejecución son más consistentes entre los distintos arreglos, con una variabilidad mínima, con excepción del arreglo de 1 000 000 elementos, con una duración de 0.355102s. Los tiempos de las corridas son bastante similares entre ellos. Los tiempos de ejecución para el algoritmo de ordenamiento por Montículos

muestran una variabilidad moderada entre las diferentes corridas. Los rangos de tiempo de ejecución son relativamente estrechos, lo que indica una consistencia general en el rendimiento del algoritmo. Los tiempos promedio son bastante estables para cada tamaño de arreglo. A medida que aumenta el tamaño del arreglo, se observa un ligero aumento gradual en el tiempo de ejecución. El algoritmo de ordenamiento Rápido muestra una variabilidad similar a la del ordenamiento por Montículos, con rangos de tiempo de ejecución similares entre las corridas de cada arreglo. Los tiempos promedio también son consistentes entre ellos, también, a medida que el tamaño del arreglo aumenta, se observa un incremento en el tiempo de ejecución, lo cual es coherente con la complejidad esperada del algoritmo. Sin embargo, en el arreglo de 1 000 000 elementos se tardó 17.7022s (en caso de Rápido) en ordenarlos, un incremento bastante alto con respecto al arreglo anterior. Por último, para el algoritmo de ordenamiento por Residuos, se observa una variabilidad similar en los tiempos de ejecución entre las diferentes corridas para cada tamaño de arreglo tanto como en los promedios. Los rangos de variación son moderados, y los tiempos consistentes. Al aumentar el tamaño del arreglo, se registra un mínimo incremento en el tiempo de ejecución. Los tiempos de ejecución de cada corrida según cada tipo de arreglo de todos los algoritmos de ordenamiento anteriormente mencionados se muestran gráficamente en las siguientes figuras 1, 2, 3, 4, 5, 6.

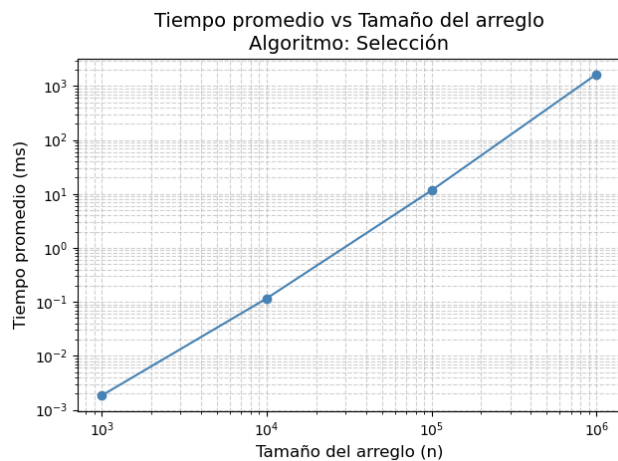


Figura 1. Tiempos promedio de ejecución del algoritmo de ordenamiento por Selección.

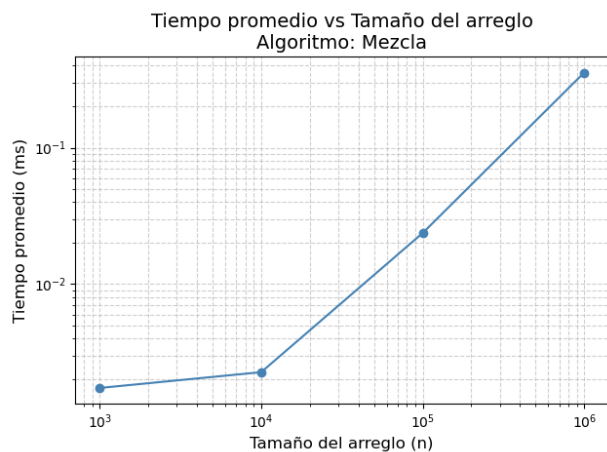
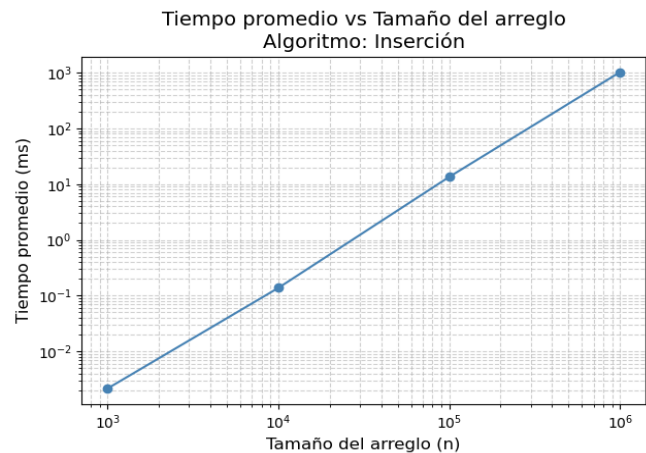


Figura 3. Tiempos promedio de ejecución del algoritmo de ordenamiento por Mezcla.

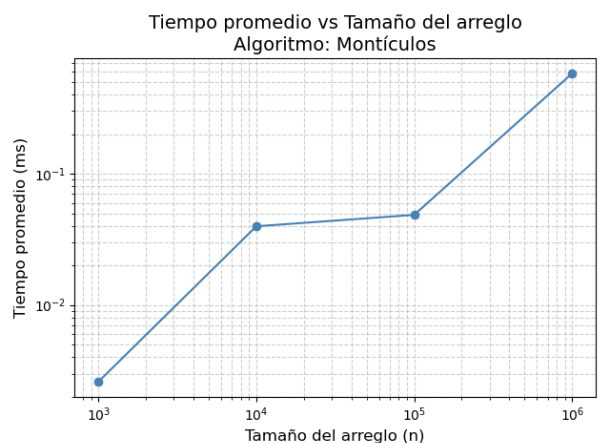


Figura 4. Tiempos promedio de ejecución del algoritmo de ordenamiento por Montículos.

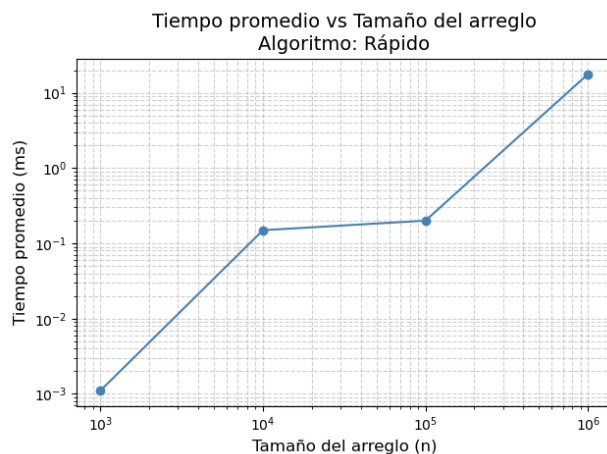


Figura 5. Tiempos promedio de ejecución del algoritmo de ordenamiento Rápido.

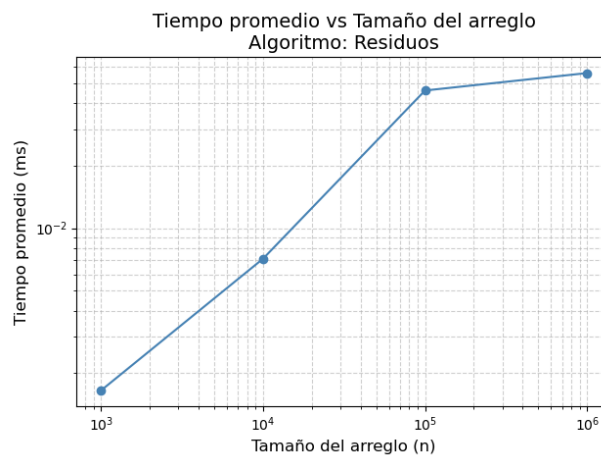


Figura 6. Tiempos promedio de ejecución del algoritmo de ordenamiento por Residuos.

Figura 2. Tiempos promedio de ejecución del algoritmo de ordenamiento por Inserción.

Selección e Inserción presentan un aumento extremadamente pronunciado del tiempo conforme crece el tamaño del arreglo y con

de memoria y facilidad de implementación al decidir qué algoritmo usar en un caso específico, lo cual puede inferir en la diferenciación entre la eficiencia práctica y teórica.

V. CONCLUSIONES

A partir de los resultados, se puede concluir que el experimento evidencia la importancia de seleccionar adecuadamente un algoritmo de ordenamiento, según las características del conjunto de datos y los requisitos de rendimiento de la aplicación. Los algoritmos por Montículos, Mezcla y Residuos destacan por su eficiencia y estabilidad, demostrando ser beneficiosos en casos con arreglos grandes o aplicaciones sensibles al tiempo de ejecución. Por otro lado, los algoritmos por Selección e Inserción, aunque sean relativamente sencillos de implementar, presentan una mayor variabilidad y menor eficiencia en arreglos grandes, lo que los hace menos adecuados para contextos donde el rendimiento es crítico. El ordenamiento Rápido también ofrece buenos tiempos en la mayoría de los casos, pero puede verse afectado por la elección del pivote, generando incrementos considerables en el tiempo de ejecución en algunos procesos. La elección del algoritmo no debe basarse únicamente en su complejidad teórica, sino también en su comportamiento empírico, incluyendo factores como la variabilidad, estabilidad, los requisitos específicos del problema a resolver, datos y resultados esperados.

Las curvas se muestran de forma conjunta en la figura 7.

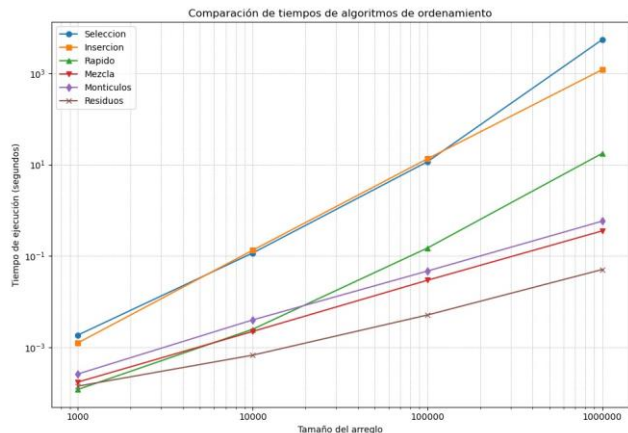


Figura 1 Gráfico comparativo de los tiempos promedio de ejecución de algoritmos de ordenamiento.

IV. DISCUSIÓN

Como primer punto, con base en los resultados presentados, se puede observar que, en la gráficas de tiempos de ejecución en escala logarítmica de cada algoritmo, se puede analizar la eficiencia relativa de los mismos; los algoritmos de ordenamiento basados en comparaciones (selección, inserción, y mezcla) muestran un aumento exponencial del tiempo de ejecución al crecer el tamaño del arreglo, de manera más drástica que los demás ordenamientos. Esto indica que pueden volverse más lentos en conjuntos de datos muy grandes. Entre ellos, Selección parece ser el menos eficiente, seguido de Inserción, ambos con complejidad $O(n^2)$, mostraron un aumento significativo en los tiempos de ejecución conforme aumentó el tamaño del arreglo que concuerda con el comportamiento esperado de las funciones cuadráticas. Su rendimiento podría ser beneficioso en arreglos parcialmente ordenados. Por otra parte, Rápido y Montículos con complejidad $O(n \log n)$ son más eficientes, con menos variabilidad en los tiempos de ejecución en los distintos arreglos y menos duración. Así mismo, Mezcla ($O(n \log n)$) y Residuos ($O(nk)$) mostraron una mayor estabilidad en los tiempos de ejecución que los anteriores, manteniendo los tiempos de ejecución de los arreglos de todos los tamaños y con aún menos duración, aunque también aumentan conforme el arreglo incrementa la cantidad de sus elementos, sin embargo, no de forma drástica sino gradual, lo que puede hacerlos más adecuados para ciertos casos con gran cantidad de datos. Los resultados experimentados coinciden con su eficiencia teórica, con una curva de crecimiento más controlada. Es importante destacar que, además del tiempo de ejecución, influyen otros factores como la estabilidad del algoritmo, uso

REFERENCIAS

- [1] Cortez Galvan, L. E. (2024). *Algoritmos de ordenamiento*. Tecnológico Nacional de México, Instituto Tecnológico de Tijuana.
- [2] Peña, H. H. M., Agudelo, A. A., & Patiño, J. I. R. (2010). Algoritmo de ordenamiento por comparaciones Heapsort. *Scientia et technica*, 17(46), 68-73.
- [3] Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1983). *Data structures and algorithms*. Addison-Wesley.
- [4] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3rd ed.). The MIT Press.



Alexa Alpízar, estudio Ingeniería en Computación, disfruto de aprender todos los días en la universidad acerca de distintos temas y me gusta mucho practicar deportes, en especial la natación artística, probar nuevos restaurantes y pasar el tiempo con mis amigos.